

CALL SHEET APPLICATION

Technical Documentation

System Architecture, Features & API Reference

Version 1.0

March 2026

Call Sheet Application — Technical Documentation

1. Introduction

The Call Sheet Application is a full-stack system for creating, editing, reviewing, approving, and publishing daily call sheets for film and television productions. It consists of three integrated components: a Node.js backend API server, a React-based web application, and a native iOS application built with SwiftUI.

Call sheets are critical production documents that communicate the day's shooting schedule, crew call times, locations, and other essential information to all cast and crew members. This application digitizes the entire call sheet lifecycle from draft creation through a two-stage approval pipeline to final publication.

1.1 Key Capabilities

- Create and edit call sheets with a structured section-based editor
- Two-stage approval workflow: Internal Review (comments) followed by Final Approval (signature)
- Real-time HTML preview with local rendering (no API calls needed)
- PDF generation for finalized call sheets
- Role-based access control (2nd AD vs other crew roles)
- Project-level metadata management with auto-incrementing shoot days
- Revision history with full payload snapshots
- Comment system for internal review stage
- Reminder system for pending approvals
- Publishing with continuation support for multi-day shoots

1.2 Technology Stack

Backend Runtime	Initially, Node.js (vanilla HTTP, no Express), now updated according to Zillit's architecture
Database	MongoDB via Mongoose ODM
Web Frontend	React 18 with Vite build tool
iOS App	SwiftUI with MVVM architecture (iOS 13.0+)

PDF Generation	Playwright (primary) with fallback PDF builder
Preview Rendering	Self-contained HTML with inline CSS
API Protocol	RESTful JSON over HTTP
ID Generation	Prefixed UUID v4 (e.g. cs_XXXX, proj_XXXX)
Timestamps	Unix epoch milliseconds (stored as Number)

2. System Architecture

The application follows a three-tier architecture with clear separation between the backend API, web client, and iOS client. All clients communicate with the backend through a RESTful JSON API.

2.1 Architecture Overview

The backend serves as the single source of truth, managing all data persistence, business logic, and status transitions. Both the web and iOS clients consume the same API endpoints, ensuring consistent behavior across platforms.

Backend Server	Vanilla Node.js HTTP server on port 3000 now updated according to Zillit's architecture
Web Client	React SPA served from /web, built with Vite (port 5176 in dev)
iOS Client	SwiftUI module with protocol-based networking
Database	MongoDB with Mongoose models for all entities
Static Assets	Client dist served from server at /web path

2.2 Backend Architecture

- **now updated according to Zillit's architecture**

2.3 iOS Architecture

The iOS app follows the MVVM (Model-View-ViewModel) pattern with reactive state management via Combine framework. Networking is abstracted behind a protocol (CallSheetNetworkProvider) allowing host apps to inject custom networking implementations.

Key iOS Design Decisions

- No force-unwraps (!) anywhere in the codebase; uses optional chaining (?.) and nil-coalescing (??) exclusively
- Local HTML rendering via CallSheetHTMLRenderer replaces all preview API calls for instant feedback
- Protocol-based networking allows the module to be embedded in any host application
- All timestamps support flexible decoding: epoch milliseconds (Double), integers, and ISO-8601 strings

2.4 Web Architecture

The web client is a React single-page application using functional components with hooks for state management. It communicates with the backend via fetch API calls and renders call sheet previews using server-generated HTML in iframes.

Key Web Components

- **App.jsx:** Root component managing editor state, tab navigation, draft/approval/published views, and modal dialogs.
- **CallSheetEditor.jsx:** Section-based editor with drag-to-reorder, inline editing, and live preview.
- **utils.js:** Shared utilities including approval history entry builder with member ID resolution.
- **Tab components:** ApprovalsTab, InternalApprovalTab, PublishedTab each handle their respective views.

3. Data Models

The application uses MongoDB for persistence with Mongoose ODM. All entities use prefixed UUID v4 identifiers and store timestamps as Unix epoch milliseconds.

3.1 Project/Metadata for Callsheet

A project represents a film or TV production. It contains metadata that controls call sheet defaults, approver lists, and shoot day tracking across all callsheets.

Field	Type	Description
id	String	Prefixed UUID (proj_xxxx)
name	String	Project name
productionCompany	String	Production company name
approverIds	String[]	Member IDs of final approvers
internalDistributionReceiverIds	String[]	Member IDs for internal distribution
totalDays	String	Total shoot days
currentShootDay	Number	Current shoot day (auto-incremented)
dayType	String	Current day type (SWD, CWD, SCWD)
dayTypes	String[]	Available day types
createdAt	Number	Unix epoch ms

3.2 Call Sheet

The core entity represents a daily call sheet. It tracks status, ownership, and links to revisions, approvals, comments, and reminders.

Field	Type	Description
-------	------	-------------

id	String	Prefixed UUID (cs_XXXX) or legacy integer
projectId	String	Parent project reference
name	String	Call sheet display name
status	String	Current lifecycle status (see Status Machine)
createdBy	String	Creator's display name
createdById	String	Creator's member ID
publishedAt	Number	Timestamp when published (null if not)
publishMeta	Object	Publishing metadata (publishedBy, continuationType, notes)
deletedAt	Number	Soft-delete timestamp (null if active)
createdAt / updatedAt	Number	Creation and last update timestamps

3.3 Revision

Each call sheet save creates a new revision with an incrementing version number. The revision stores the complete payload snapshot, ensuring full history preservation.

Field	Type	Description
id	String	Prefixed UUID (rev_XXXX)
callSheetId	String	Parent call sheet reference
version	Number	Sequential version number (1, 2, 3...)
createdBy / createdById	String	Author name and member ID
payload	Object	Complete call sheet payload (sections model)
notes	String	Optional revision notes
createdAt	Number	Unix epoch ms

3.4 Approval Request

Represents an individual approval action assigned to a specific approver. Multiple requests are created per approval round (one per approver).

Field	Type	Description
id	String	Prefixed UUID (aprq_XXXX)
callSheetId	String	Call sheet being approved
stage	String	INTERNAL or FINAL
round	Number	Approval round number
assigneeId / assigneeName	String	Approver identity
role	String	Approver's role/designation
status	String	PENDING, APPROVED, REJECTED, or SUPERSEDED
reason	String	Rejection reason (null if approved)
signatureImage	String	Base64 signature (for FINAL stage)
actedAt	Number	Timestamp of approval/rejection action

3.5 Project Member (No need - dummy while developing)

Represents a crew member associated with a project. Members can be designated as approvers and are used for name resolution throughout the system.

Field	Type	Description
id	String	Prefixed UUID (mem_XXXX)
projectId	String	Parent project
fullName	String	Display name
department	String	Department (e.g. Production, Camera)
designation	String	Role (e.g. 2nd Assistant Director, Director)
isApprover	Boolean	Whether this member is a designated approver
createdAt	Number	Unix epoch ms

3.6 Other Entities

Comment

Comments are used during the Internal Approval stage for reviewers to provide feedback on the call sheet.

- **Fields:** id (cmt_XXXX), callSheetId, authorId, authorName, authorRole, text, createdAt

Reminder

Reminders are sent by the call sheet creator to pending approvers to nudge them toward action.

- **Fields:** id (rem_xxxx), callSheetId, approvalRequestId, assigneeId, assigneeName, sentBy, sentByRole, message, createdAt

4. Call Sheet Payload Structure

The call sheet payload uses a sections-model format that organizes content into a hierarchy of page rows, cells, and data rows. This structure enables flexible layout with multiple sections per row and various rendering types.

4.1 Payload Hierarchy

```
payload
├─ shared (SharedData)
│   ├── shootDayNumber, totalDays, date, dayType
│   ├── approverIds[], internalReceiverIds[]
│   └─ currentScript, currentSchedule
└─ page_rows[] (PageRow)
    ├── order, break (page break flag)
    └─ page_row_cells[] (Cell)
        ├── section_title, type, render_as
        ├── row_header[] (column definitions)
        └─ rows[] (DataRow)
            └─ row[] (CellValue: value, css_value, attachment)
```

4.2 Cell Types

Type	Render As	Description
section	generic	Key-value info display (e.g. location, producer). Renders as labeled rows.
section	employee	Department member listing with call times. Special 'In' column formatting.
table	generic	Grid layout with column headers and data rows. Fixed table layout.
notes	(any)	Bulleted or horizontal list of notes. Title bar with note items below.

4.3 Special Rendering Rules

- Top Sections: First page row with 3 cells renders as a 3-column grid (executives, company, call times)
- Employee sections: 'In' column strips 'Time:' and 'Other:' prefixes for cleaner display
- Users column: Comma-separated member IDs are resolved to full names via member lookup
- Date fields: Epoch milliseconds converted to 'DD Mon YYYY' format

- Time fields: Epoch milliseconds converted to 'HH:MM' 24-hour format
- Radio channels: Special grid layout with 8-column repeating rows
- Approvers section: Rendered from project metadata (authoritative) or payload approverIds (fallback)

4.4 Default Template

New call sheets are created from a default template that includes standard production sections:

- Executives Names (Director, Producer, Writer, 1st AD)
- Company Details (Project Name, Company, Address, Onset)
- Call Times (Script, Schedule, Shooting Call, Parking, Catering)
- Crew departments with call time and location columns
- Additional notes section

The template's shared data is auto-populated from project metadata: shoot day number (current + 1), total days, day type, and approver/receiver IDs.

5. Approval Workflow

The application implements a two-stage approval pipeline designed for film production workflows. The first stage (Internal) is for team review and comments. The second stage (Final) is for executive sign-off with optional digital signatures.

5.1 Status State Machine

Call sheets progress through a defined set of statuses:

Status	Display Name	Description
DRAFT	Draft	Initial state. Editable by creator.
UNDER_COMMENTS	For Comments	Sent for internal team comments.
PENDING_INTERNAL_APPROVAL (not in scope anymore)	For Comments	Submitted for internal approval (comment stage).
INTERNAL_APPROVAL_REJECTED (not in scope anymore)	Comments Rejected	Rejected during internal review.
INTERNAL_APPROVED (not in scope anymore)	Comments Approved	All internal approvers have been approved.
PENDING_APPROVAL	Pending Signature	Submitted for final executive approval.
APPROVAL_REJECTED	Final Rejected	Rejected during final approval.
APPROVED_FOR_PUBLISH	Final Approved	All final approvers signed off.
PUBLISHED	Published	Published and distributed to the crew.
DELETED	Deleted	Soft-deleted (preserves data).

5.2 Stage 1: Internal Approval (Comments)

Purpose: Allow team members to review the call sheet and provide comments before executive sign-off.

- Creator selects team members as internal reviewers from the project's internal distribution list
- Recipients receive approval requests with stage = INTERNAL
- Reviewers can add comments via the Comments system
- Each reviewer can Approve or Reject the call sheet (**not in scope anymore**)
- When ALL internal approvers approve, the system automatically creates FINAL stage approval requests (if project has configured finalApproverIds) (**not in scope anymore**)
- If any reviewer rejects, status becomes INTERNAL_APPROVAL_REJECTED (**not in scope anymore**)

5.3 Stage 2: Final Approval (Signature)

Purpose: Executive sign-off with optional digital signature capture.

- Final approvers are determined by project metadata (finalApproverIds), not the call sheet creator
- Each approver can approve with a digital signature image, approve without signature, or reject with a reason
- When ALL final approvers approve, status becomes APPROVED_FOR_PUBLISH
- If any approver rejects, status becomes APPROVAL_REJECTED

5.4 Re-submission Behavior

When a call sheet is re-submitted for approval (e.g., after edits following a rejection):

- Old PENDING approval requests from prior rounds are marked as SUPERSEDED (not deleted)
- This preserves the full audit trail of all approval actions
- A new round number is assigned, incrementing from the previous highest round
- A 2-second grace period prevents race conditions during round transitions

5.5 Publishing

After final approval, the creator can publish the call sheet:

- Only the original creator can publish their call sheet
- Requires status = APPROVED_FOR_PUBLISH
- Supports two continuation types: NEW (first call sheet) or CONTINUATION (follow-up to a previous day)
- CONTINUATION requires publish notes and a valid previous published call sheet in the same project
- Publishing records: publishedBy, publishedAt, continuationType, continuationOfCallSheetId, publishNotes

6. API Reference

All API endpoints accept and return JSON. The server runs on port 3000 by default (configurable via PORT environment variable). The web client is served at /web.

6.1 Call Sheet Endpoints

Method	Path	Description
POST	/call-sheets	Create new call sheet with initial revision

GET	/call-sheets	List call sheets (filter by status, projectId, createdBy, createdById, approverId)
GET	/call-sheets/:id	Get full call sheet detail with approvals, reminders, revisions
DELETE	/call-sheets/:id	Soft-delete call sheet (sets status=DELETED)

6.2 Revision Endpoints

Method	Path	Description
GET	/call-sheets/:id/revisions	List all revisions (newest first)
POST	/call-sheets/:id/revisions	Create new revision (resets status to DRAFT)

6.3 Approval Endpoints

Method	Path	Description
POST	/call-sheets/:id/submit-for-approval	Submit for final approval
POST	/call-sheets/:id/submit-for-internal-approval	Submit for internal review
POST	/call-sheets/:id/send-for-comments	Transition to UNDER_COMMENTS
GET	/approval-requests	List approval requests (filter by assigneeId, status, stage)
POST	/approval-requests/:id/approve	Approve request (optional signatureImage)
POST	/approval-requests/:id/reject	Reject request (optional reason)

6.4 Preview & PDF Endpoints

Method	Path	Description
GET	/call-sheets/:id/preview	Full HTML preview of saved call sheet
POST	/call-sheets/:id/preview	Live preview with unsaved payload
GET	/call-sheets/:id/preview/section	Single cell preview (query: rowIdx, cellIdx)

POST	/preview/from-payload	Render HTML from raw payload (no ID needed)
GET	/call-sheets/:id/pdf	Generate PDF (Playwright or fallback)

6.5 Other Endpoints

Method	Path	Description
POST	/projects	Create project
GET	/projects	List all projects
GET	/projects/:id/call-sheet-metadata	Read project shoot settings
PUT	/projects/:id/call-sheet-metadata	Update metadata (propagates to call sheets)
POST	/projects/:id/members	Add project member
GET	/projects/:id/members	List project members
GET	/call-sheets/:id/comments	List comments
POST	/call-sheets/:id/comments	Add comment
POST	/call-sheets/:id/send-reminder	Send approval reminder
GET	/call-sheets/:id/reminders	List reminders
POST	/call-sheets/:id/publish	Publish call sheet
GET	/default-template	Get default call sheet template
GET	/weather/today	Fetch weather summary (query: location)
GET	/health	Health check

7. Web Application

The web client is a React SPA that provides the full call sheet management experience in a browser. It is built with Vite and serves as static files from the backend at the /web path.

7.1 Navigation Flow

```
Project List
├─ Project Detail (tabbed view)
│   └─ Drafts Tab
│       ├── Draft list with edit/delete/send actions
│       ├── Create new call sheet (from template)
│       └─ Call Sheet Editor (full-screen)
│           └─ Live HTML preview in iframe
```

- └─ Approvals Tab (Sent / Received / Finalized sub-tabs)
 - | └─ Approve/Reject with signature or reason
 - | └─ View history, comments, reminders
- └─ Published Tab
 - └─ Latest + older published call sheets

7.2 Editor Features

- Section-based editing: tap any section to edit its title, headers, and data rows
- Drag-to-reorder page rows via up/down buttons
- Add new sections: section, table, notes, or page break
- Insert sections between existing ones via + markers
- Edit shared information: shoot day, total days, date, day type
- Live HTML preview rendered in an iframe alongside the editor
- Unsaved changes detection with confirmation dialog on back navigation
- Save behavior: existing sheets save directly; new sheets prompt for draft name first

7.3 Approval Management

- Sent tab: Shows call sheets created by user with approval status summaries
- Received tab: Shows pending approval requests for the current user
- Finalized tab: Shows APPROVED_FOR_PUBLISH call sheets
- Internal approval: Comments panel with read/write access
- Final approval: Signature capture or approve-without-signature option
- History modal: Shows full revision and approval history with member name resolution

7.4 Member ID Resolution (not in our system's scope)

The web app resolves member IDs (prefixed with 'mem_') to human-readable names throughout the UI. The `approvalHistoryEntries()` utility in `utils.js` builds a member map and uses a `resolveName()` helper to convert any member ID references to full names in history entries.

8. iOS Application

The iOS client is built as a SwiftUI module (`CallSheetModule`) that can be embedded in a host application. It follows MVVM architecture with Combine for reactive state management and targets iOS 13.0+.

8.1 Module Structure

Directory	Contents	Purpose
Models/	CallSheet, Payload, Status, Member, Project, etc.	Data model definitions with Codable conformance
Networking/	NetworkProvider protocol, Service, Endpoint builder	Abstracted API communication layer
ViewModels/	Editor, Detail, Lists, Approvals, Comments VMs	Business logic and state management

Views/	SwiftUI views, sheets, components	UI layer with navigation and interaction
Utilities/	HTMLRenderer, DateFormatter, Colors, AnyCodable	Shared utilities and helpers

8.2 Navigation Flow

```

CallSheetRootView (entry point)
└─ ProjectListView (project carousel + user picker)
    └─ ProjectDetailView (tabbed: Drafts / Approvals / Published) The actual Callsheet tool
        └─ DraftsTabContent
            └─ DraftListView (view/edit/delete/send)
            └─ EditorLoaderView → CallSheetEditorView
                └─ SharedInfoEditorSheet
                └─ CellEditorSheet
                └─ CallSheetPreviewEditorView (live HTML)
            └─ SendForApprovalSheet (2-step flow)
        └─ ApprovalsTabContent
            └─ ApprovalListView (Sent/Received/Finalized)
        └─ PublishedTabContent
            └─ PublishedListView

```

8.3 Local HTML Rendering

The iOS app renders call sheet previews entirely on-device using `CallSheetHTMLRenderer.swift`. This eliminates API calls for preview, providing instant visual feedback as the user edits.

Renderer Capabilities

- Full page preview: complete call sheet rendered as self-contained HTML
- Section preview: single cell rendered in isolation for editor context
- Member ID to name resolution using the project's member list
- Date/time formatting (epoch ms to human-readable)
- Special column handling: 'In' column prefix stripping, 'Users' column ID resolution
- Approvers section rendered from project metadata (authoritative source)
- Interactive tap targets with data attributes for section/shared/approver editing
- Visual touch feedback with opacity transitions
- Horizontally scrollable preview with 960px minimum width for comfortable viewing

8.4 Protocol-Based Networking

The networking layer is abstracted behind the `CallSheetNetworkProvider` protocol, allowing the host application to inject its own networking implementation (e.g., with authentication headers, custom SSL pinning, or logging).

- `CallSheetNetworkProvider`: Protocol defining `request<T>` and `requestData` methods
- `DefaultNetworkProvider`: URLSession-based implementation for standalone use
- `CallSheetService`: Convenience layer with typed methods for all API operations
- `APIEndpoint`: Builder pattern for constructing endpoint requests with path, method, query, and body
- Combine extensions: `requestPublisher<T>` and `requestDataPublisher` for reactive code

8.5 Key iOS Features

- **Unsaved changes dialog:** When navigating back from the editor with pending edits, an alert prompts Save or Discard
- **Smart save routing:** Existing sheets save directly; new sheets prompt for a name first
- **Shoot day auto-increment:** On first save, if the auto-filled shoot day was not changed, metadata's currentShootDay is incremented
- **Role-based tabs:** 2nd AD sees Drafts + Approvals + Published; others see Approvals + Published only
- **Flexible timestamp decoding:** Handles epoch ms (Double), integers, and ISO-8601 strings seamlessly
- **History with name resolution:** Approval history resolves member IDs to display names

9. Project Metadata & Configuration

Project metadata controls default values for new call sheets and manages the approval pipeline configuration. Changes to metadata propagate automatically to all active call sheets in the project.

9.1 Metadata Fields

Field	Description	Propagation
current_shoot_day	Current shoot day number	Auto-incremented on call sheet save
total_days	Total planned shoot days	Applied to new call sheet shared data
day_type	Current day type (SWD/CWD/SCWD)	Applied to new call sheet shared data
day_types	Available day type options	Merged with defaults: SWD, CWD, SCWD
finalApproverIds	Member IDs for final approval	Propagated to ALL non-deleted call sheets
internal_distribution_receivers	Member IDs for internal review	Propagated to ALL non-deleted call sheets

9.2 Approver Propagation

When finalApproverIds or internal_distribution_receivers are updated in project metadata, the backend automatically propagates these changes to all non-deleted call sheets in the project. This ensures consistency: the project metadata is always the authoritative source for who can approve.

- Updates are applied to the latest revision's payload of each call sheet
- Both the revision payload and the call sheet's updatedAt timestamp are modified
- The HTML renderer always prefers metadata approvers over payload approvers

9.3 Shoot Day Auto-Increment

When creating a new call sheet, the system auto-fills the shoot day number as currentShootDay + 1. On first save:

- If the user did not change the auto-filled value, the system updates currentShootDay in metadata
- This ensures the next new call sheet gets the correct sequential day number

- If the user manually changed the shoot day, no metadata update occurs
- The total_days field is also synced to metadata if present

10. Feature Summary

10.1 Core Features

Feature	Web	iOS	Description
Create Call Sheet	Yes	Yes	From default template with auto-filled metadata
Edit Sections	Yes	Yes	Tap/click sections to edit title, headers, rows
Reorder Sections	Yes	Yes	Move page rows up/down
Insert Sections	Yes	Yes	Add sections between existing ones via + markers
Live Preview	Yes	Yes	Real-time HTML preview (iframe / WKWebView)
PDF Generation	Yes	No	Server-side PDF via Playwright
Save Drafts	Yes	Yes	Save with revision history
Unsaved Changes	Yes	Yes	Confirmation dialog on back navigation
Draft Naming	Yes	Yes	Name prompt for new call sheets

10.2 Approval Features

Feature	Web	iOS	Description
Send for Comments	Yes	Yes	Submit to internal reviewers
Internal Approval (not in scope anymore)	Yes	Yes	Approve/reject during comment stage
Send for Signature	Yes	Yes	Submit to final approvers

Digital Signature	Yes	Yes	Capture signature image on approval
Rejection with Reason	Yes	Yes	Provide rejection reason
Comments	Yes	Yes	Add/view comments during internal review
Reminders	Yes	Yes	Send reminders to pending approvers
History	Yes	Yes	Full approval and revision history
Re-submission	Yes	Yes	Re-submit after edits (SUPERSEDED tracking)
Publishing	Yes	Yes	Publish with continuation support

10.3 Management Features

Feature	Web	iOS	Description
Project Management (no need in our system)	Yes	Yes	Create and list projects
Member Management (no need in our system)	Yes	Yes	Add members with department/designation
Metadata Management	Yes	Yes	Configure shoot days, approvers, receivers
User Selection (no need in our system)	Yes	Yes	Switch between users/members
Role-Based Access	Yes	Yes	2nd AD sees all tabs; others see approvals only
Soft Delete	Yes	Yes	Delete drafts (preserves data)
Weather Integration	Yes	No	Fetch today's weather for shoot location

11. Conventions & Standards

11.1 ID Format

All entities use prefixed UUID v4 identifiers for clear identification:

Prefix	Entity	Example
proj_	Project	proj_a1b2c3d4-e5f6-7890-abcd-ef1234567890
mem_	Member	mem_d4ffa21-8b3c-4e2a-9f1b-a-bc123def456
cs_	Call Sheet	cs_12345678-abcd-4321-efgh-ijklmnopqrst
rev_	Revision	rev_98765432-wxyz-1234-abcd-ef9876543210
aprq_	Approval Request	aprq_abcdef12-3456-7890-abcd-ef1234567890
cmt_	Comment	cmt_11223344-5566-7788-9900-aabbccddeeff
rem_	Reminder	rem_ffeeddcc-bbaa-9988-7766-554433221100

11.2 Timestamp Convention

All timestamps are stored as Unix epoch milliseconds (JavaScript `Date.now()` format). The iOS app supports flexible decoding from epoch ms (Double), integer, and ISO-8601 string formats. Display formatting converts to human-readable strings like 'DD Mon YYYY' for dates and 'HH:MM' for times.

11.3 API Response Format

All list endpoints wrap results in a named array:

```
{ "projects": [...] }
{ "callSheets": [...] }
{ "approvalRequests": [...] }
{ "members": [...] }
```

Detail endpoints return a single object:

```
{ "callSheet": {...} }
{ "project": {...} }
```

11.4 Error Handling

The API returns standard HTTP status codes with JSON error bodies:

- 400 Bad Request: Missing required fields or invalid input
- 404 Not Found: Entity not found by ID
- 409 Conflict: Status transition not allowed
- 500 Internal Server Error: Unhandled server exceptions (caught by error boundary)

--- End of Documentation ---

Page